

CS43900-IDV - Fall 2026

Homework Assignment 1

Visualizing Tabular Data : Comparing EVs

Key Dates

Handed out:	Thursday, August 27, 2026
Due date:	Thursday, September 10, 2026 (before 11:59 pm)

Objectives

This first homework gives you a quick introduction to several visualization techniques that are popular in practice. In particular, it familiarizes you with *Matplotlib/pyplot*, which you will use this semester. **Note:** Please refer to the syllabus for explanations of what constitutes an **authorized use of LLMs/AI** in the completion of this homework.

Context

The dataset for this homework gives the specifications of various electric vehicles introduced in the US market between 2010 and 2024. It is a tabular dataset where the items (or *keys*) correspond to the individual car models, and the attributes (or *values*) indicate important characteristics. This type of dataset can be challenging to visualize because showing all the attributes (9 in this case) at once is virtually impossible. Here, you will tackle this problem with different strategies.

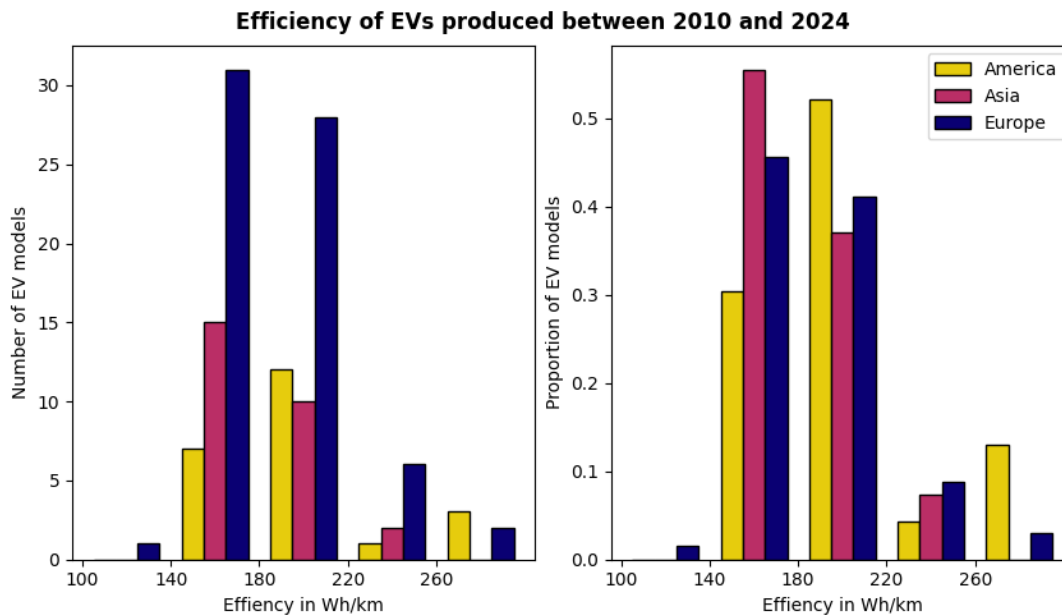
Specifically, the dataset indicates for each vehicle:

- "Manufacturer" (text)
- "Model" (text)
- "Weight" (numerical value, in kg)
- "Top Speed" (numerical value, in km/h)
- "Range" (numerical value in km)
- "Acceleration" (numerical value in sec)
- "Efficiency" (numerical value in Wh/km)
- "Price" (numerical value, in euros)
- "Country" / "Region": geographic production origin (text)

There are 118 entries in this table, stored in an Excel workbook.

Tasks

For each task, use `argparse` to implement the required API.



Task 1: Grouped Bar Chart

Visualize the efficiency distribution of all models by production region using a grouped bar chart (see above, left). In other words, you will

1. **Split** the range of efficiency values between 100 Wh/km and 280 Wh/km in 40 Wh/km-wide intervals (there will be five intervals: [100, 140), [140, 180), [180, 220), [220, 260), [260-280)).
2. For each efficiency interval, **count** how many cars from each production origin fall in the interval.
3. Visualize the corresponding counts as **bars**. For each interval, there will be 3 bars (America, Asia, Europe). Pick a **color** for each production origin to color all the bars associated with that origin.
4. Include color legends, labels, and titles.

You will observe that this dataset contains many more EV models produced in Europe than in America or Asia (again, we are looking at the US market). For that reason, the 'Europe' bars in the previous visualization tend to obscure the other bars. To correct this effect, we can instead visualize the **relative fraction** of the EVs produced in each region that fall in each interval. For instance, the distribution of American EVs in the intervals listed above is (0, 7, 12, 1, 3). There are 23 cars in total. Therefore, the relative fractions of US EVs per interval are (0/23=0%, 7/23=30%, 12/23=52%, 3/23=13%). Create a second grouped bar chart to show this distribution for all three

geographic origins. Include both grouped bar charts in a single window, showing them side by side. Include legends, axes labels, and title.

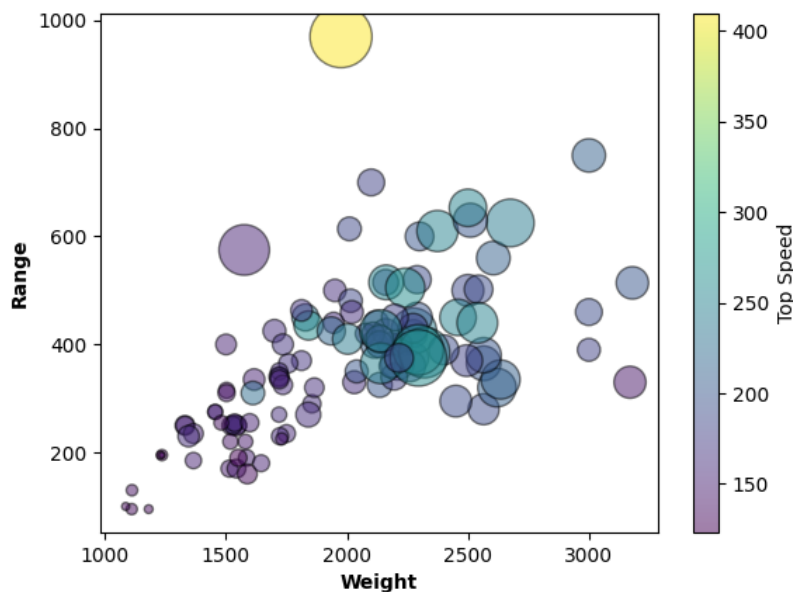
Deliverable: hw1_bars.py

API: `python hw1_bars.py -i <filename>`

where <filename> is the data file's name (including path).

Task 2: Bubble Chart

Bubble chart representation of Weight, Range, Price, Top Speed



The previous visualization showed only the EVs' efficiency attributes. It would be interesting to visualize the relationship that exists between several characteristics. To do so, you will use a technique called **bubble chart** in which each car will be visualized according to 4 different attributes. Assume that these four attributes are called a_1 , a_2 , a_3 , and a_4 . Each car will be represented by a **circle** centered at $(x=a_1, y=a_2)$, with an *area* corresponding to a_3 , and a color that represents a_4 . Representing a value as a color is done using a **color map**. Use one of the **perceptually uniform color maps** provided by Matplotlib in your implementation. Include **legend**, **labels**, **color bar**, and **title**. You will omit the legend for the size of each circle since it is difficult to create with Matplotlib. We will revisit that problem in the next homework.

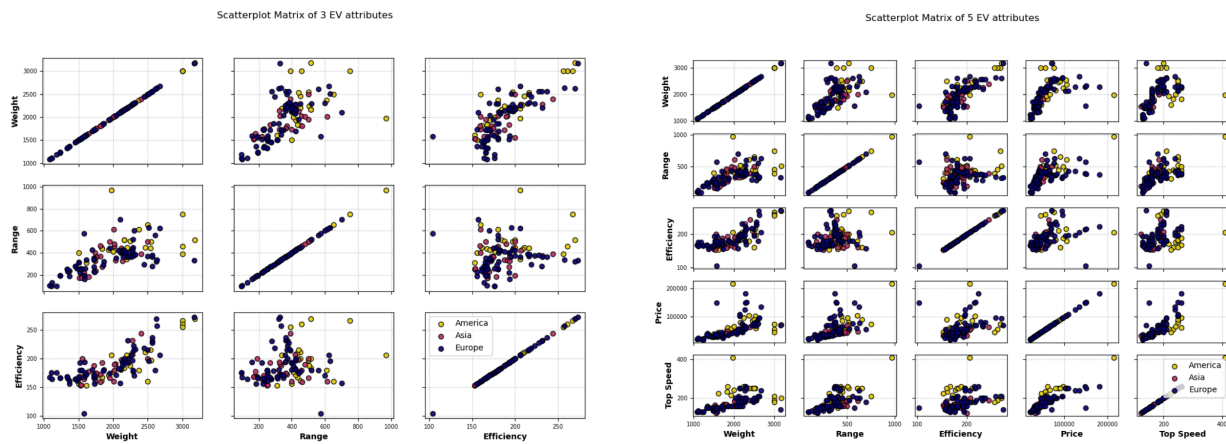
Deliverable: hw1_bubbles.py

API: `python hw1_bubbles.py -i <filename> -x <attribute> -y <attribute> -c <attribute> -s <attribute>`

where <filename> is the data file's name (including path), and the remaining parameters indicate what attributes to associate with the degrees of freedom (x, y, color, size) of each circle. These attributes should be identified by their name in the spreadsheet.

The above image shows a sample result for x='Weight', y='Range', s='Price', c='Top Speed' using the "[Viridis](#)" color map.

Task 3: Scatter Plot Matrix



While bubble charts can show 4 quantities simultaneously, one may want to inspect the correlation that exists between a larger number of attributes or adopt a more quantitative perspective than what size and color can afford. A solution to this problem is to use a scatter plot matrix ("***splo***m" for short). A scatterplot matrix is a 2D array of scatterplots: each row of the matrix assigns a different attribute to the vertical axis of all the scatterplots contained in that row. There are, therefore, as many rows as the number of attributes to plot, and all the plots in a given row share the same vertical axis. Similarly, every column assigns a different attribute to the horizontal axis of all the scatterplots in that column, and all the plots in a given column share the same horizontal axis. Usually, one uses the same set of attributes for rows and columns, and the matrix is square. Practically, considering N attributes (*), you should:

1. Create N columns corresponding to N different attributes (*) from the dataset
2. Create N rows corresponding to the same attributes
3. For each entry of the resulting array, create a scatterplot for the pair of attributes associated with that row and column.
4. Color the individual points by the production region, using the same color conventions as in Task 1.
5. Include legends, labels, and a title.

(*): Note that only numerical attributes can be used to define coordinate axes.

Deliverable: hw1_splom.py

API: `python hw1_splom.py -i <filename> -a <attribute 1> -a <attribute 2> -a <attribute 3> [... -a <attribute N>]`

where <filename> is the data file's name (including path), and each following -a <attribute> indicates an attribute to include in the splom. Note that your program should be able to take up to 7 attribute names (where 7 is the number of numerical attributes in the dataset) to construct the splom.

The above image shows you possible results for 3 and 5 attributes, respectively.

Data Set

The dataset is available in Excel workbook format on Brightspace under **Contents/Datasets/Homework1: EVs**. It is derived from a dataset available on [Kaggle](#), which I extended using internet scraping. I make no guarantee as to the accuracy of the information.

Submission

Submit your results for this project on *Gradescope* “**Assignment 1: Comparing Electric Vehicles**” before **Thursday, September 10, 2026, at 11:59 pm**. Refer to the instructions below.

- Include all the requested Python files: **hw1_bars.py**, **hw1_bubbles.py**, and **hw1_splom.py**.
- Use Python 3.12 (or a more recent version) in your implementation as well as version 3.10 of matplotlib (or a more recent version).
- **Do not include the data file** in your submission.
- **Reminder:** Check that your programs satisfy the prescribed API. In particular, **test them on the command line** (not inside an IDE or Jupyter) and make sure that they properly handle the path of the data file as a command-line parameter.
- **Reminder:** Indicate how you used AI by filling out the form that you will find on Gradescope under “**Assignment 1: AI Declaration**”

Last modified: **August 27, 2026**.